DAM 0485 · Programación · Unidad Didáctica 4

Boletín 4.1: Nivel Inicial (Resuelto) — Soluciones Comentadas

Soluciones completas y comentadas de los 16 ejercicios de nivel inicial de la Unidad 4 en Java 25 LTS.

A continuación se presentan las soluciones comentadas de los 16 ejercicios iniciales de la Unidad 4, implementadas siguiendo los estándares de encapsulación, validación defensiva y buenas prácticas de Java 25 LTS.

Solución Ejercicio 1: La Clase `Libro`, Delegación con `this(...)` y Encapsulamiento Básico

```
public class Libro {
    private String titulo;
    private String autor;
    private int numeroPaginas;

    // Constructor canónico con validación
    public Libro(String titulo, String autor, int numeroPaginas) {
        setTitulo(titulo);
        setAutor(autor);
        setNumeroPaginas(numeroPaginas);
    }

    // Constructor sin parámetros que delega en el constructor principal mediante this(...)
    public Libro() {
        this("Sin título", "Desconocido", 0);
    }

    public String getTitulo() {
        return this.titulo;
    }

    public void setTitulo(String titulo) {
        if (titulo == null || titulo.trim().isEmpty()) {
            throw new IllegalArgumentException("El título del libro no puede ser nulo ni vac
ío.");
        }
        this.titulo = titulo.trim();
    }

    public String getAutor() {
        return this.autor;
    }

    public void setAutor(String autor) {
        if (autor == null || autor.trim().isEmpty()) {
            throw new IllegalArgumentException("El autor del libro no puede ser nulo ni vací
o.");
        }
        this.autor = autor.trim();
    }

    public int getNumeroPaginas() {
        return this.numeroPaginas;
    }

    public void setNumeroPaginas(int numeroPaginas) {
        if (numeroPaginas < 0) {
            throw new IllegalArgumentException("El número de páginas no puede ser negativo:
" + numeroPaginas);
        }
        this.numeroPaginas = numeroPaginas;
    }

    public boolean esLargo() {
        return this.numeroPaginas > 300;
    }
}
```

```
public class MainLibro {
    public static void main(String[] args) {
        Libro l1 = new Libro("Ficciones", "Borges", 200);
        Libro l2 = new Libro("1984", "Orwell", 328);
        Libro l3 = new Libro(); // "Sin título", "Desconocido", 0

        System.out.printf("Libro 1: '%s' de %s (%d págs.) -> ¿Es largo? %b%n",
                l1.getTitulo(), l1.getAutor(), l1.getNumeroPaginas(), l1.esLargo());
        System.out.printf("Libro 2: '%s' de %s (%d págs.) -> ¿Es largo? %b%n",
                l2.getTitulo(), l2.getAutor(), l2.getNumeroPaginas(), l2.esLargo());
        System.out.printf("Libro 3 (por defecto): '%s' de %s (%d págs.)%n",
                l3.getTitulo(), l3.getAutor(), l3.getNumeroPaginas());

        // Modificamos el número de páginas de l1 con su setter
        l1.setNumeroPaginas(224);
        System.out.println("Páginas actualizadas de '" + l1.getTitulo() + "': " + l1.getNume
roPaginas());
    }
}
```

Solución Ejercicio 2: Geometría y Contador Estático de Instancias con `Rectangulo`

```
public class Rectangulo {
    private static int numeroDeRectangulos = 0;

    private double base;
    private double altura;

    public Rectangulo(double base, double altura) {
        if (base <= 0 || altura <= 0) {
            throw new IllegalArgumentException("La base y la altura deben ser mayores que ce
ro.");
        }
        this.base = base;
        this.altura = altura;
        numeroDeRectangulos++;
    }

    public static int getNumeroDeRectangulos() {
        return numeroDeRectangulos;
    }

    public double getBase() {
        return this.base;
    }

    public void setBase(double base) {
        if (base <= 0) throw new IllegalArgumentException("La base debe ser positiva.");
        this.base = base;
    }

    public double getAltura() {
        return this.altura;
    }

    public void setAltura(double altura) {
        if (altura <= 0) throw new IllegalArgumentException("La altura debe ser positiva.");
        this.altura = altura;
    }

    public double calcularArea() {
        return this.base * this.altura;
    }

    public double calcularPerimetro() {
        return 2 * (this.base + this.altura);
    }

    public boolean esCuadrado() {
        return Double.compare(this.base, this.altura) == 0;
    }

    public void escalar(double factor) {
        if (factor <= 0) {
            throw new IllegalArgumentException("El factor de escala debe ser positivo: " + f
actor);
        }
        this.base *= factor;
        this.altura *= factor;
    }

    public static void main(String[] args) {
```

```
        Rectangulo r1 = new Rectangulo(4, 3);
        Rectangulo r2 = new Rectangulo(5, 5);
        Rectangulo r3 = new Rectangulo(10, 2);

        System.out.println("Área r1: " + r1.calcularArea() + " | Perímetro r1: " + r1.calcul
arPerimetro());
        System.out.println("¿r2 es cuadrado?: " + r2.esCuadrado());
        System.out.println("Total de rectángulos creados: " + Rectangulo.getNumeroDeRectangu
los()); // 3
    }
}
```

Solución Ejercicio 3: La Clase `Circulo`

```
public class Circulo {
    private double radio;

    public Circulo(double radio) {
        if (radio <= 0) {
            throw new IllegalArgumentException("El radio del círculo debe ser estrictamente
positivo.");
        }
        this.radio = radio;
    }

    // Constructor sin parámetros delegando en el canónico
    public Circulo() {
        this(1.0);
    }

    public double getRadio() {
        return this.radio;
    }

    public void setRadio(double radio) {
        if (radio <= 0) throw new IllegalArgumentException("Radio no válido.");
        this.radio = radio;
    }

    public double calcularArea() {
        return Math.PI * Math.pow(this.radio, 2);
    }

    public double calcularPerimetro() {
        return 2 * Math.PI * this.radio;
    }
}
```

Solución Ejercicio 4: Conversor Térmico con la Clase `Termometro`

```
public class Termometro {
    public static final double CERO_ABSOLUTO_CELSIUS = -273.15;
    private double temperaturaCelsius;

    public Termometro(double temperaturaCelsius) {
        setTemperaturaCelsius(temperaturaCelsius);
    }

    public double getTemperaturaCelsius() {
        return this.temperaturaCelsius;
    }

    public void setTemperaturaCelsius(double celsius) {
        if (celsius < CERO_ABSOLUTO_CELSIUS) {
            throw new IllegalArgumentException("Temperatura por debajo del cero absoluto: "
+ celsius);
        }
        this.temperaturaCelsius = celsius;
    }

    public double getFahrenheit() {
        return (this.temperaturaCelsius * 9.0 / 5.0) + 32.0;
    }

    public double getKelvin() {
        return this.temperaturaCelsius + 273.15;
    }

    public void incrementar(double delta) {
        setTemperaturaCelsius(this.temperaturaCelsius + delta);
    }

    public void decrementar(double delta) {
        setTemperaturaCelsius(this.temperaturaCelsius - delta);
    }
}
```

Solución Ejercicio 5: Coordenadas Cartesianas con `Punto2D`

```
public class Punto2D {
    private double x;
    private double y;

    public Punto2D(double x, double y) {
        this.x = x;
        this.y = y;
    }

    public Punto2D() {
        this(0.0, 0.0);
    }

    public double getX() { return this.x; }
    public void setX(double x) { this.x = x; }

    public double getY() { return this.y; }
    public void setY(double y) { this.y = y; }

    public void desplazar(double dx, double dy) {
        this.x += dx;
        this.y += dy;
    }

    public double distanciaAlOrigen() {
        return Math.sqrt(Math.pow(this.x, 2) + Math.pow(this.y, 2));
    }
}
```

Solución Ejercicio 6: La Máquina Cafetera

```
public class Cafetera {
    private int capacidadMaxima;
    private int cantidadActual;

    public Cafetera() {
        this(1000, 0);
    }

    public Cafetera(int capacidadMaxima) {
        this(capacidadMaxima, 0);
    }

    public Cafetera(int capacidadMaxima, int cantidadActual) {
        this.capacidadMaxima = Math.max(capacidadMaxima, 100);
        if (cantidadActual > this.capacidadMaxima) {
            this.cantidadActual = this.capacidadMaxima;
        } else {
            this.cantidadActual = Math.max(cantidadActual, 0);
        }
    }

    public void llenarCafetera() {
        this.cantidadActual = this.capacidadMaxima;
    }

    public int servirTaza(int capacidadTaza) {
        if (capacidadTaza <= 0) return 0;

        if (this.cantidadActual >= capacidadTaza) {
            this.cantidadActual -= capacidadTaza;
            return capacidadTaza;
        } else {
            int servido = this.cantidadActual;
            this.cantidadActual = 0;
            return servido; // Sirve lo que queda
        }
    }

    public void vaciarCafetera() {
        this.cantidadActual = 0;
    }

    public void agregarCafe(int cantidad) {
        if (cantidad > 0) {
            this.cantidadActual = Math.min(this.cantidadActual + cantidad, this.capacidadMax
ima);
        }
    }

    public int getCapacidadMaxima() { return this.capacidadMaxima; }
    public int getCantidadActual() { return this.cantidadActual; }
}
```

Solución Ejercicio 7: Cronómetro Simple con `Reloj`

```
public class Reloj {
    private int hora;
    private int minuto;
    private int segundo;

    public Reloj(int hora, int minuto, int segundo) {
        if (hora < 0 || hora > 23 || minuto < 0 || minuto > 59 || segundo < 0 || segundo > 5
9) {
            throw new IllegalArgumentException("Valores de tiempo fuera de límites.");
        }
        this.hora = hora;
        this.minuto = minuto;
        this.segundo = segundo;
    }

    public Reloj() {
        this(0, 0, 0);
    }

    public void ticTac() {
        this.segundo++;
        if (this.segundo == 60) {
            this.segundo = 0;
            this.minuto++;
            if (this.minuto == 60) {
                this.minuto = 0;
                this.hora++;
                if (this.hora == 24) {
                    this.hora = 0;
                }
            }
        }
    }

    public String getHoraFormateada() {
        return String.format("%02d:%02d:%02d", this.hora, this.minuto, this.segundo);
    }
}
```

Solución Ejercicio 8: Identificadores Autoincrementales y Sueldo Protegido con `Empleado`

```
import java.util.Objects;

public class Empleado {
    private static int contadorSecuencial = 1000;
    private static String passwordMaestra = "RRHH2025";

    private final int idEmpleado;
    private String nombre;
    private int edad;
    private String departamento;
    private double sueldo;

    public Empleado(String nombre, int edad, String departamento, double sueldo) {
        setNombre(nombre);
        setEdad(edad);
        this.idEmpleado = ++contadorSecuencial;
        this.departamento = (departamento != null) ? departamento : "General";
        this.sueldo = Math.max(sueldo, 0);
    }

    public static int getTotalEmpleados() {
        return contadorSecuencial - 1000;
    }

    public int getIdEmpleado() { return this.idEmpleado; }
    public String getNombre() { return this.nombre; }

    public void setNombre(String nombre) {
        if (nombre == null || nombre.trim().isEmpty()) {
            throw new IllegalArgumentException("El nombre del empleado es obligatorio.");
        }
        this.nombre = nombre.trim();
    }

    public int getEdad() { return this.edad; }

    public void setEdad(int edad) {
        if (edad < 18) {
            throw new IllegalArgumentException("El empleado debe ser mayor de edad (>= 18):
" + edad);
        }
        this.edad = edad;
    }

    public double getSueldo(String password) {
        verificarPassword(password);
        return this.sueldo;
    }

    public void setSueldo(double nuevoSueldo, String password) {
        verificarPassword(password);
        if (nuevoSueldo < 0) {
            throw new IllegalArgumentException("El sueldo no puede ser negativo.");
        }
        this.sueldo = nuevoSueldo;
    }

    private static void verificarPassword(String password) {
        if (!Objects.equals(passwordMaestra, password)) {
            throw new SecurityException("Acceso denegado: contraseña de RRHH incorrecta.");
```

```
        }
    }
}
```

Solución Ejercicio 9: La Clase `Contador` y Encadenamiento

```
public class Contador {
    private int valor;

    public Contador() {
        this(0);
    }

    public Contador(int valorInicial) {
        this.valor = valorInicial;
    }

    public Contador(Contador otro) {
        this(otro != null ? otro.valor : 0);
    }

    public Contador incrementar() {
        this.valor++;
        return this; // Permite method chaining
    }

    public Contador decrementar() {
        this.valor--;
        return this;
    }

    public Contador incrementar(int cantidad) {
        this.valor += cantidad;
        return this;
    }

    public int getValor() {
        return this.valor;
    }
}
```

Solución Ejercicio 10: Array de Objetos Alumnos

```
public class GestionAlumnos {
    public static class Alumno {
        private final String nombre;
        private final double nota;

        public Alumno(String nombre, double nota) {
            this.nombre = nombre;
            this.nota = nota;
        }

        public String getNombre() { return this.nombre; }
        public double getNota() { return this.nota; }
    }

    public static double calcularNotaMedia(Alumno[] lista) {
        if (lista == null || lista.length == 0) return 0.0;
        double suma = 0;
        for (Alumno a : lista) {
            suma += a.getNota();
        }
        return suma / lista.length;
    }

    public static Alumno obtenerMejorAlumno(Alumno[] lista) {
        if (lista == null || lista.length == 0) return null;
        Alumno mejor = lista[0];
        for (int i = 1; i < lista.length; i++) {
            if (lista[i].getNota() > mejor.getNota()) {
                mejor = lista[i];
            }
        }
        return mejor;
    }

    public static void main(String[] args) {
        Alumno[] clase = {
            new Alumno("Elena", 8.5),
            new Alumno("Marcos", 6.2),
            new Alumno("Sara", 9.8),
            new Alumno("David", 7.0),
            new Alumno("Lucía", 5.4)
        };

        System.out.printf("Nota media de la clase: %.2f%n", calcularNotaMedia(clase));
        Alumno top = obtenerMejorAlumno(clase);
        System.out.printf("Mejor alumno: %s con un %.1f%n", top.getNombre(), top.getNota());
    }
}
```

Solución Ejercicio 11: Control de Vehículo con la Clase `Coche`

```
public class Coche {
    private final String marca;
    private final String modelo;
    private boolean motorEncendido;
    private int velocidadActual;

    public Coche(String marca, String modelo) {
        this.marca = marca;
        this.modelo = modelo;
        this.motorEncendido = false;
        this.velocidadActual = 0;
    }

    public boolean arrancar() {
        if (this.motorEncendido) return false;
        this.motorEncendido = true;
        return true;
    }

    public boolean apagar() {
        if (!this.motorEncendido || this.velocidadActual > 0) {
            return false; // Peligro: no apagar en marcha
        }
        this.motorEncendido = false;
        return true;
    }

    public void acelerar(int incremento) {
        if (this.motorEncendido && incremento > 0) {
            this.velocidadActual = Math.min(this.velocidadActual + incremento, 200);
        }
    }

    public void frenar(int decremento) {
        if (decremento > 0) {
            this.velocidadActual = Math.max(this.velocidadActual - decremento, 0);
        }
    }

    public int getVelocidadActual() { return this.velocidadActual; }
    public boolean isMotorEncendido() { return this.motorEncendido; }
}
```

Solución Ejercicio 12: Generador y Validador de Contraseñas

```
import java.security.SecureRandom;

public class Password {
    private static final String CARACTERES = "ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrst
uvwxyz0123456789";
    private static final SecureRandom RANDOM = new SecureRandom();

    private int longitud;
    private String valor;

    public Password() {
        this(8);
    }

    public Password(int longitud) {
        this.longitud = Math.max(longitud, 4);
        regenerar();
    }

    public void regenerar() {
        StringBuilder sb = new StringBuilder(this.longitud);
        for (int i = 0; i < this.longitud; i++) {
            int idx = RANDOM.nextInt(CARACTERES.length());
            sb.append(CARACTERES.charAt(idx));
        }
        this.valor = sb.toString();
    }

    public boolean esFuerte() {
        int mayus = 0;
        int minus = 0;
        int digitos = 0;

        for (int i = 0; i < this.valor.length(); i++) {
            char c = this.valor.charAt(i);
            if (Character.isUpperCase(c)) mayus++;
            else if (Character.isLowerCase(c)) minus++;
            else if (Character.isDigit(c)) digitos++;
        }

        return mayus >= 2 && minus >= 1 && digitos >= 2;
    }

    public String getValor() { return this.valor; }
    public int getLongitud() { return this.longitud; }
}
```

Solución Ejercicio 13: Visibilidad `private` vs. `public` y Auditoría de Código con Errores

1. Diferencia entre `private` y `public`:

- Un atributo **`public`** es accesible y modificable directamente desde cualquier clase externa del programa (`libro.paginas = -50;`), dejando al objeto indefenso frente a estados inválidos y creando un fuerte acoplamiento con su estructura interna.
- Un atributo **`private`** solo es visible dentro de la propia clase que lo declara. Combinado con métodos públicos `get` y `set` (la **interfaz pública**), permite validar que los datos nunca tomen valores incoherentes

(principio de **encapsulamiento** y ocultación de la información).

2. Auditoría 1 (`public static String getNombre() { return nombre; }`):

- **No compila** (`non-static variable nombre cannot be referenced from a static context`).
- **Razón conceptual:** `nombre` es un atributo de instancia (cada empleado tiene su propio nombre en el Heap), mientras que un método `static` pertenece a la clase global y se ejecuta sin objeto (`this` no existe). La clase no puede adivinar el nombre de qué empleado concreto quieres devolver.

3. Auditoría 2 (Corrección de `Punto2D`):

- *Errores detectados en el código original:* `Public` con P mayúscula (error léxico), la clase se llamaba `empleado` en el fichero `Punto2D.java`, atributos `public` sin encapsular, el supuesto constructor `public void Punto2D(...)` tenía tipo de retorno `void` (convirtiéndolo en un método normal, no en un constructor), y `x = x; y = y;` asignaba los parámetros a sí mismos por sombreamiento al omitir `this.x = x;`.
- *Código corregido:*

```
public class Punto2D {
    private double x;
    private double y;

    public Punto2D(double x, double y) {
        this.x = x;
        this.y = y;
    }

    public double getX() { return this.x; }
    public double getY() { return this.y; }
}
```

Solución Ejercicio 14: Traza de Referencias Compartidas en el Heap

Salida por pantalla:

```
El Aleph
true
```

Explicación paso a paso de la memoria:

1. `new Libro("Ficciones", "Borges", 200)` reserva un **único objeto** en el **Heap** y guarda su dirección en la variable `l1` del **Stack**.
2. `Libro l2 = l1;` **no duplica el objeto**, sino que copia la dirección de memoria de `l1` en `l2`. Ambas referencias apuntan exactamente al mismo objeto físico en el Heap.
3. Al ejecutar `l2.setTitulo("El Aleph")`, se modifica el atributo `titulo` de ese único objeto compartido. Por eso `l1.getTitulo()` devuelve `"El Aleph"` y `l1 == l2` evalúa a `true`.
4. **Cómo crear una copia independiente:** Tendríamos que instanciar un nuevo objeto en el Heap con `new`, ya sea pasando los getters (`Libro l2 = new Libro(l1.getTitulo(), l1.getAutor(), l1.getNumeroPaginas());`) o implementando un **constructor de copia** `public Libro(Libro otro)`.

Solución Ejercicio 15: La Clase `FormateadorBanner`: Encapsulamiento y Sobrecarga Orientada a Objetos

```
public class FormateadorBanner {
    private char caracterBorde;
    private int anchoMinimo;
    private static int totalBannersGenerados = 0;

    // Constructor canónico con validación
    public FormateadorBanner(char caracterBorde, int anchoMinimo) {
        if (anchoMinimo < 10) {
            throw new IllegalArgumentException("El ancho mínimo no puede ser menor a 10.");
        }
        this.caracterBorde = caracterBorde;
        this.anchoMinimo = anchoMinimo;
    }

    // Constructor por defecto delegando con this(...)
    public FormateadorBanner() {
        this('*', 30);
    }

    // 1. Método sobrecargado principal
    public void imprimirBanner(String texto) {
        imprimirBanner(texto, this.caracterBorde);
    }

    // 2. Sobrecarga con carácter especial
    public void imprimirBanner(String texto, char caracterEspecial) {
        String textoLimpio = (texto == null) ? "" : texto.trim();
        int longitudLinea = Math.max(this.anchoMinimo, textoLimpio.length() + 4);

        imprimirLineaBorde(longitudLinea, caracterEspecial);
        System.out.printf("%c %s %c%n", caracterEspecial, centrarTexto(textoLimpio, longitud
Linea - 4), caracterEspecial);
        imprimirLineaBorde(longitudLinea, caracterEspecial);

        totalBannersGenerados++;
    }

    // 3. Método con varargs para múltiples líneas
    public void imprimirLineas(String... lineas) {
        if (lineas == null || lineas.length == 0) {
            imprimirBanner("");
            return;
        }
        int maxLen = 0;
        for (String linea : lineas) {
            if (linea != null && linea.length() > maxLen) {
                maxLen = linea.length();
            }
        }
        int longitudLinea = Math.max(this.anchoMinimo, maxLen + 4);

        imprimirLineaBorde(longitudLinea, this.caracterBorde);
        for (String linea : lineas) {
            String l = (linea == null) ? "" : linea;
            System.out.printf("%c %-" + (longitudLinea - 4) + "s %c%n", this.caracterBorde,
l, this.caracterBorde);
        }
        imprimirLineaBorde(longitudLinea, this.caracterBorde);

        totalBannersGenerados++;
```

```
    }

    // Métodos auxiliares privados para no duplicar lógica
    private void imprimirLineaBorde(int longitud, char c) {
        for (int i = 0; i < longitud; i++) {
            System.out.print(c);
        }
        System.out.println();
    }

    private String centrarTexto(String texto, int espacio) {
        int padding = (espacio - texto.length()) / 2;
        StringBuilder sb = new StringBuilder();
        for (int i = 0; i < padding; i++) sb.append(' ');
        sb.append(texto);
        while (sb.length() < espacio) sb.append(' ');
        return sb.toString();
    }

    public static int getTotalBannersGenerados() {
        return totalBannersGenerados;
    }

    public static void main(String[] args) {
        FormateadorBanner fb1 = new FormateadorBanner();
        FormateadorBanner fb2 = new FormateadorBanner('=', 40);

        fb1.imprimirBanner("BIENVENIDOS A 1º DAM");
        fb1.imprimirBanner("AVISO URGENTE", '!');
        fb2.imprimirLineas("SISTEMA DE GESTIÓN", "Módulo de Programación", "Java 25 LTS");

        System.out.println("Total de banners generados: " + FormateadorBanner.getTotalBanner
sGenerados());
    }
}
```

Solución Ejercicio 16: Modelado de Objetos con Estado: Las Clases `Dado` y `Cubilete`

```
import java.util.Random;

public class SimuladorDadosPOO {

    // 1. CLASE DADO: Entidad con estado y comportamiento propio
    public static class Dado {
        private final int numeroCaras;
        private int ultimoValor;
        private static final Random RANDOM = new Random();

        public Dado(int numeroCaras) {
            if (numeroCaras < 2) {
                throw new IllegalArgumentException("Un dado debe tener como mínimo 2 cara
s.");
            }
            this.numeroCaras = numeroCaras;
            this.ultimoValor = 1;
        }

        public Dado() {
            this(6); // Dado estándar de 6 caras
        }

        public int lanzar() {
            this.ultimoValor = RANDOM.nextInt(this.numeroCaras) + 1;
            return this.ultimoValor;
        }

        public int getNumeroCaras() { return numeroCaras; }
        public int getUltimoValor() { return ultimoValor; }

        @Override
        public String toString() {
            return String.format("Dado de %d caras (última tirada: %d)", numeroCaras, ultimo
Valor);
        }
    }

    // 2. CLASE CUBILETE: Gestiona un array de objetos Dado
    public static class Cubilete {
        private final Dado[] dados;

        public Cubilete(int cantidadDados, int carasPorDado) {
            if (cantidadDados <= 0) {
                throw new IllegalArgumentException("Debe haber al menos 1 dado en el cubilet
e.");
            }
            this.dados = new Dado[cantidadDados];
            for (int i = 0; i < cantidadDados; i++) {
                this.dados[i] = new Dado(carasPorDado);
            }
        }

        public int tirarTodos() {
            int suma = 0;
            for (Dado d : this.dados) {
                suma += d.lanzar();
            }
            return suma;
        }
```

```
        public int tirarHastaSuma(int sumaObjetivo) {
            int minimoPosible = this.dados.length;
            int maximoPosible = 0;
            for (Dado d : this.dados) maximoPosible += d.getNumeroCaras();

            if (sumaObjetivo < minimoPosible || sumaObjetivo > maximoPosible) {
                throw new IllegalArgumentException(String.format(
                        "Suma imposible. Rango válido: [%d..%d]", minimoPosible, maximoPosib
le));
            }

            int tiradas = 0;
            int suma;
            do {
                suma = tirarTodos();
                tiradas++;
            } while (suma != sumaObjetivo);

            return tiradas;
        }

        public void mostrarEstado() {
            for (int i = 0; i < dados.length; i++) {
                System.out.printf("  [Dado %d] %s%n", i + 1, dados[i]);
            }
        }
    }

    public static void main(String[] args) {
        // Prueba de un dado suelto de 20 caras (D20 para rol)
        Dado d20 = new Dado(20);
        System.out.println("Tirada D20: " + d20.lanzar());
        System.out.println(d20);

        // Prueba del cubilete con 2 dados de 6 caras
        Cubilete cubilete = new Cubilete(2, 6);
        System.out.println("\n--- Tirada inicial del cubilete ---");
        int sumaInicial = cubilete.tirarTodos();
        System.out.println("Suma obtenida: " + sumaInicial);
        cubilete.mostrarEstado();

        // Tirar hasta obtener un 12 (doble 6)
        System.out.println("\nLanzando hasta conseguir un 12...");
        int intentos = cubilete.tirarHastaSuma(12);
        System.out.printf("¡Conseguido en %d tiradas!%n", intentos);
        cubilete.mostrarEstado();
    }
}
```